

Name: Manahil Habib

Reg no: 2023-BSE-042

Section: 5B

Lab 09

Task 1 — GitHub CLI, Codespace setup and authentication

Goal: Install GH CLI (if not already present), authenticate with GitHub, create a Codespace, and connect to it. All work for the rest of the lab must be done inside the Codespace.

Steps (do these from your local machine shell first, then run codespace commands):

1.(Local desktop) Install GitHub CLI (Windows example via winget):

winget install --id GitHub.cli

```
C:\Users\HABIB TARIQ>gh --version
gh version 2.83.2 (2025-12-10)
https://github.com/cli/cli/releases/tag/v2.83.2
```

2. (Local) Authenticate GH CLI for Codespaces:

gh auth login -s codespace

When prompted, generate a GitHub Access Token (classic) in the browser with the following Token scopes:

admin:org

codespace

repo

Copy the token into the GH CLI prompt to complete login.

```

C:\Users\HABIB TARIQ>gh auth login -s codespace
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: C4AF-3C7C
Press Enter to open https://github.com/login/device in your browser...
✓ Authentication complete.
- gh config set -h github.com git_protocol https
✓ Configured git protocol
✓ Logged in as manahilhabib031
! You were already logged in to this account

```

3.(Local) List available Codespaces (optional verification):

gh codespace list

```

C:\Users\HABIB TARIQ>gh codespace list
NAME          DISPLAY...  REPOSITORY  BRANCH  STATE    CREATED AT
expert-...    expert ...  manahil...  main*   Shutdown about 8...

```

4. (Local) Create or connect to a Codespace. To create a new codespace from the current repo:

gh codespace create --repo <owner>/<repo> --branch main --machine basicLinux32gb

```

C:\Users\HABIB TARIQ>gh codespace create --repo manahilhabib031/Lab9 --branch main --machine basicLinux32gb
✓ Codespaces usage for this repository is paid for by manahilhabib031
crispy-rotary-phone-5g7q7575664pcp477

```

Or to open an existing codespace:

gh codespace ssh -c <name_of_codespace>

After connecting via gh codespace ssh, you will be inside a Linux shell that the rest of this lab assumes.

```
C:\Windows\system32>gh codespace ssh -c special-space-umbrella-r77v55j5v9j3xgwj
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.8.0-1030-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

@23-22411-071-eng  /workspaces/my-lab-repo (main) $
```

Task 2 — Install AWS CLI inside the Codespace and configure it

Goal: Install the AWS CLI in the Codespace and configure it to interact with your AWS account.

Inside the Codespace shell run:

1.Download and install AWS CLI:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
```

```
unzip awscliv2.zip
```

```
sudo ./aws/install
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0     0     0     0     0      0     0 --  62 60.2M    62 37.7M    0
  0 65.9M     0 --100 60.2M  100 60.2M    0     0 81.3M    0 --:--:-- --:
--:-- --:--:-- 81.3M
@manahilhabib031 → /workspaces/Lab9 (main) $
```

```

@manahilhabib031 → /workspaces/Lab9 (main) $ sudo apt update && sudo apt install unzip -y
Get:1 https://packages.microsoft.com/repos/microsoft-ubuntu-noble-prod noble InRelease [3600 B]
Hit:2 https://repo.anaconda.com/pkg/misc/debrepo/conda stable InRelease
Hit:3 https://dl.yarnpkg.com/debian stable InRelease
Get:4 https://packages.microsoft.com/repos/microsoft-ubuntu-noble-prod noble/main amd64 Packages
8 kB]
Hit:5 http://archive.ubuntu.com/ubuntu noble InRelease
Get:6 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1950 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1754 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [2130 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1185 kB]
Fetched 7480 kB in 4s (1845 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
20 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
unzip is already the newest version (6.0-28ubuntu4.1).
0 upgraded, 0 newly installed, 0 to remove and 20 not upgraded.
@manahilhabib031 → /workspaces/Lab9 (main) $

```

```

@manahilhabib031 → /workspaces/Lab9 (main) $ sudo ./aws/install
Found preexisting AWS CLI installation: /usr/local/aws-cli/v2/current. Please rerun install script wi
th --update flag.
@manahilhabib031 → /workspaces/Lab9 (main) $

```

2. Verify installation:

aws --version

```

@manahilhabib031 → /workspaces/Lab9 (main) $ aws --version
aws-cli/2.32.26 Python/3.13.11 Linux/6.8.0-1030-azure exe/x86_64.ubuntu.24
@manahilhabib031 → /workspaces/Lab9 (main) $

```

3. Configure the AWS CLI (you will provide Access Key ID and Secret Access Key for a user with permissions, or use root/IAM user you prepared for the lab):

aws configure

Enter:

AWS Access Key ID

AWS Secret Access Key

Default region name (e.g., me-central-1)

Default output format (e.g., json)

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws configure
AWS Access Key ID [*****RUKG]: AKIARMIC7HAZCXVX
AWS Secret Access Key [*****N2OS]: qQB0RxHL36ala
ds05hMW

Default output format [json]:
@manahilhabib031 → /workspaces/Lab9 (main) $
```

4. Verify credentials/config files:

```
cat ~/.aws/credentials
```

```
cat ~/.aws/config
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ cat ~/.aws/credentials
[default]
aws_access_key_id = AKIAPMTCT7
aws_secret_access_key = qQB0RxHL36ala
ds05hMW
@manahilhabib031 → /workspaces/Lab9 (main) $ cat ~/.aws/config
[default]
region = me-central-1
output = json
@manahilhabib031 → /workspaces/Lab9 (main) $
```

5. Verify connectivity (you should see a JSON result showing your caller identity):

```
aws sts get-caller-identity
```

Expected output (example):

```
{
  "UserId": "EXAMPLEUSERID1234567890",
  "Account": "123456789012",
  "Arn": "arn:aws:iam::123456789012:user/ExampleUser"
}
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws sts get-caller-identity
{
  "UserId": "AIDAPMTCT7",
  "Account": "0950323",
  "Arn": "arn:aws:iam::0950323:user/labcc"
}
@manahilhabib031 → /workspaces/Lab9 (main) $
```

Task 3 — Create security group and add ingress rules using Codespace IP

Goal: Create an EC2 security group, inspect it, add an SSH rule for your Codespace public IP, and verify.

Steps (run in the Codespace shell):

1. Create a security group (substitute your VPC id):

```
aws ec2 create-security-group --group-name 'MySecurityGroup' \  
--description 'My Security Group' \  
--vpc-id 'vpc-EXAMPLE1234567890'
```

This command returns a security group id, for example sg-EXAMPLE1234567890.

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 create-security-group \  
--group-name 'MySecurityGroup' \  
--description 'My Security Group' \  
--vpc-id 'vpc-021a5e129ca204b3f' \  
{  
  "GroupId": "sg-04c021947ffac853a",  
  "SecurityGroupArn": "arn:aws:ec2:me-central-1:095032391730:security-group/sg-04c021947ffac853a"  
}
```

2. Inspect the security group (initially ingress will be empty or default):

```
aws ec2 describe-security-groups --group-ids sg-EXAMPLE1234567890
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-security-groups --group-ids sg-04c021947ffac853a  
{  
  "SecurityGroups": [  
    {  
      "GroupId": "sg-04c021947ffac853a",  
      "IpPermissionsEgress": [  
        {  
          "IpProtocol": "-1",  
          "UserIdGroupPairs": [],  
          "IpRanges": [  
            {  
              "CidrIp": "0.0.0.0/0"  
            }  
          ],  
          "Ipv6Ranges": [],  
          "PrefixListIds": []  
        }  
      ],  
      "VpcId": "vpc-021a5e129ca204b3f",  
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:095032391730:security-group/sg-04c021947ffac853a",  
      "OwnerId": "095032391730",  
      "GroupName": "MySecurityGroup",  
      "Description": "My Security Group",  
      "IpPermissions": []  
    }  
  ]  
}
```

3. Get your Codespace public IP (from inside the Codespace):

curl icanhazip.com

```
@manahilhabib031 → /workspaces/Lab9 (main) $ curl icanhazip.com
4.240.39.193
@manahilhabib031 → /workspaces/Lab9 (main) $
```

4. Authorize SSH inbound on port 22 from your Codespace IP:

aws ec2 authorize-security-group-ingress \

--group-id sg-EXAMPLE1234567890 \

--protocol tcp \

--port 22 \

--cidr <XXX.XXX.XXX.XXX>/32

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 authorize-se
curity-group-ingress \
  --group-id sg-04c021947ffac853a \
  --protocol tcp \
  --port 22 \
  --cidr 4.240.39.193/32
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-056ecf5acd81bf827",
      "GroupId": "sg-04c021947ffac853a",
      "GroupOwnerId": "095032391730",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 22,
      "ToPort": 22,
      "CidrIpv4": "4.240.39.193/32",
      "SecurityGroupRuleArn": "arn:aws:ec2:me-central-1:095
032391730:security-group-rule/sgr-056ecf5acd81bf827"
    }
  ]
}
```

5. Verify ingress rule appears:

aws ec2 describe-security-groups --group-ids sg-EXAMPLE1234567890

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-security-groups --group-ids sg-04c021947ffac853a
{
  "SecurityGroups": [
    {
      "GroupId": "sg-04c021947ffac853a",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-021a5e129ca204b3f",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:095032391730:security-group/sg-04c021947ffac853a",
      "OwnerId": "095032391730",
      "GroupName": "MySecurityGroup",
      "Description": "My Security Group",
      "IpPermissions": [
        {
          "IpProtocol": "tcp",
          "FromPort": 22,
          "ToPort": 22,
          "UserIdGroupPairs": [],
          "IpRanges": [
            {

```

6. Add an HTTP rule (port 80) using ip-permissions JSON:

```
aws ec2 authorize-security-group-ingress \
```

```
--group-id 'sg-EXAMPLE1234567890' \
```

```
--ip-permissions
```

```
'{"FromPort":80,"ToPort":80,"IpProtocol":"tcp","IpRanges":[{"CidrIp":"<XXX.XXX.XXX.XXX>/32"}]}'
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 authorize-security-group-ingress \
--group-id sg-04c021947ffac853a \
--ip-permissions '{"FromPort":80,"ToPort":80,"IpProtocol":"tcp","IpRanges":[{"CidrIp":"4.240.39.193/32"}]}'
{
  "Return": true,
  "SecurityGroupRules": [
    {
      "SecurityGroupRuleId": "sgr-0c168eea835a8f262",
      "GroupId": "sg-04c021947ffac853a",
      "GroupOwnerId": "095032391730",
      "IsEgress": false,
      "IpProtocol": "tcp",
      "FromPort": 80,
      "ToPort": 80,
      "CidrIpv4": "4.240.39.193/32",
      "SecurityGroupRuleArn": "arn:aws:ec2:me-central-1:095032391730:security-group-rule/sgr-0c168eea835a8f262"
    }
  ]
}
```


7. Verify both ingress rules are present:

aws ec2 describe-security-groups --group-ids sg-EXAMPLE1234567890

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-security-groups --group-ids sg-04c021947ffac853a
{
  "SecurityGroups": [
    {
      "GroupId": "sg-04c021947ffac853a",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-021a5e129ca204b3f",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:095032391730:security-group/sg-04c021947ffac853a",
      "OwnerId": "095032391730",
      "GroupName": "MySecurityGroup",
      "Description": "My Security Group",
      "IpPermissions": [
        {
          "IpProtocol": "tcp",
          "FromPort": 80,
          "ToPort": 80,
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "4.240.39.193/32"
            }
          ]
        }
      ]
    }
  ]
}
```

Task 4 — Create a key pair, describe key pairs, and launch EC2 instance

Goal: Create an ED25519 key pair, view it, and launch an EC2 instance using the key pair and the security group created earlier.

Steps:

1. Create the key pair and save the PEM file into the Codespace workspace:

aws ec2 create-key-pair \

--key-name MyED25519Key \

--key-type ed25519 \

--key-format pem \

--query 'KeyMaterial' \

--output text > MyED25519Key.pem

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 create-key-pair \
--key-name MyED25519Key \
--key-type ed25519 \
--key-format pem \
--query 'KeyMaterial' \
--output text > MyED25519Key.pem
@manahilhabib031 → /workspaces/Lab9 (main) $ ls -l MyED25519Key.pem
-rw-rw-rw- 1 codespace codespace 388 Jan  9 02:01 MyED25519Key.pem
@manahilhabib031 → /workspaces/Lab9 (main) $
```

2.View created key pairs:

aws ec2 describe-key-pairs

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-key-pairs
{
  "KeyPairs": [
    {
      "KeyPairId": "key-0acde7469035039ae",
      "KeyType": "rsa",
      "Tags": [],
      "CreateTime": "2026-01-08T10:29:41.268000+00:00",
      "KeyName": "myweb",
      "KeyFingerprint": "b4:a7:52:2d:1a:e6:8a:35:d3:93:0d:52:1a:82:8b:86:cf:fb:d7:ad"
    },
    {
      "KeyPairId": "key-087e258ebae802e06",
      "KeyType": "ed25519",
      "Tags": [],
      "CreateTime": "2026-01-09T02:01:39.262000+00:00",
      "KeyName": "MyED25519Key",
      "KeyFingerprint": "+/v0daq+pMvGvTaCIU2N4XnL80mQB4SVIObZzJJzgF4="
    }
  ]
}
@manahilhabib031 → /workspaces/Lab9 (main) $
```

3.(Do not) Delete key pair:

aws ec2 delete-key-pair --key-name MyED25519Key # Info: shows how to delete

4.Launch an EC2 instance (example values — replace IDs with ones from your account/region):

aws ec2 run-instances \

--image-id ami-05e66df2bafcb7dea \

```
--count 1 \  
--instance-type t3.micro \  
--key-name MyED25519Key \  
--security-group-ids sg-EXAMPLE1234567890 \  
--subnet-id subnet-EXAMPLE1234567890 \  
--tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=MyServer}]"
```

```

@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 run-instances \
--image-id ami-05e66df2bafcb7dea \
--count 1 \
--instance-type t3.micro \
--key-name MyED25519Key \
--security-group-ids sg-04c021947ffac853a \
--subnet-id subnet-0f284e40a191a3e62 \
--tag-specifications "ResourceType=instance,Tags=[{Key=Name,Value=MyServer}]"

{
  "ReservationId": "r-0cf816d9114b40dfc",
  "OwnerId": "095032391730",
  "Groups": [],
  "Instances": [
    {
      "Architecture": "x86_64",
      "BlockDeviceMappings": [],
      "ClientToken": "045eb5cb-cb2f-4435-81a4-237c478bc78c",
      "EbsOptimized": false,
      "EnaSupport": true,
      "Hypervisor": "xen",
      "NetworkInterfaces": [
        {
          "Attachment": {
            "AttachTime": "2026-01-09T02:06:21+00:00",
            "AttachmentId": "eni-attach-0f77452bab1cfc87c",
            "DeleteOnTermination": true,
            "DeviceIndex": 0,
            "Status": "attaching",
            "NetworkCardIndex": 0
          },
          "Description": "",
          "Groups": [
            {
              "GroupId": "sg-04c021947ffac853a",
              "GroupName": "MySecurityGroup"
            }
          ],
          "Ipv6Addresses": [],
          "MacAddress": "0e:cc:6e:1e:15:d7",
          "NetworkInterfaceId": "eni-033a72f79c6bb9f7a",
          "OwnerId": "095032391730",
          "PrivateDnsName": "ip-172-31-3-43.me-central-1.compute.internal",
          :
        }
      ],
      "ReservationId": "r-0cf816d9114b40dfc",
      "OwnerId": "095032391730",
      "Groups": [],

```

5. Get the public IP address of your instance:

```
aws ec2 describe-instances \
```

```
--query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress]" \
```

```
--output table
```

```

@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-instances \
--query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress]" \
--output table
-----
| DescribeInstances |
+-----+-----+
| i-080271458b4e9b2d1 | 3.28.200.35 |
| i-0ffbcddcd6910d18a4 | 158.252.21.171 |
+-----+-----+
@manahilhabib031 → /workspaces/Lab9 (main) $

```

6. Attempt SSH into the instance from the Codespace or from a machine whose IP is allowed in the security group:

```
ssh -i MyED25519Key.pem ec2-user@<public-ip-address>
```

If you see the error:

Permissions 0644 for 'MyED25519Key.pem' are too open. It is required that your private key files are NOT accessible by others.

fix permissions:

```
chmod 400 MyED25519Key.pem
```

```
ssh -i MyED25519Key.pem ec2-user@<public-ip-address>
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 stop-instances --instance-ids i-0ffbc
cdcd6910d18a4
{
  "StoppingInstances": [
    {
      "InstanceId": "i-0ffbcddcd6910d18a4",
      "CurrentState": {
        "Code": 64,
        "Name": "stopping"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 terminate-instances --instance-ids i-0ffbcddcd6910d18a4
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 start-instances --instance-ids i-0ffbcdcd6910d18a4
{
  "StartingInstances": [
    {
      "InstanceId": "i-0ffbcdcd6910d18a4",
      "CurrentState": {
        "Code": 0,
        "Name": "pending"
      },
      "PreviousState": {
        "Code": 80,
        "Name": "stopped"
      }
    }
  ]
}
```

Task 5 — Understand AWS describe-* commands

Goal: Use describe commands to list and inspect AWS resources.

1.Run and understand these commands (run each, then capture screenshot immediately after):

aws ec2 describe-security-groups

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-security-groups
{
  "SecurityGroups": [
    {
      "GroupId": "sg-071d1c28d597d6865",
      "IpPermissionsEgress": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [],
          "IpRanges": [
            {
              "CidrIp": "0.0.0.0/0"
            }
          ],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ],
      "VpcId": "vpc-021a5e129ca204b3f",
      "SecurityGroupArn": "arn:aws:ec2:me-central-1:095032391730:security-group/sg-071d1c28d597d6865",
      "OwnerId": "095032391730",
      "GroupName": "default",
      "Description": "default VPC security group",
      "IpPermissions": [
        {
          "IpProtocol": "-1",
          "UserIdGroupPairs": [
            {
              "UserId": "095032391730",
              "GroupId": "sg-071d1c28d597d6865"
            }
          ],
          "IpRanges": [],
          "Ipv6Ranges": [],
          "PrefixListIds": []
        }
      ]
    }
  ]
}
```

aws ec2 describe-vpcs

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-vpcs
{
  "Vpcs": [
    {
      "OwnerId": "095032391730",
      "InstanceTenancy": "default",
      "CidrBlockAssociationSet": [
        {
          "AssociationId": "vpc-cidr-assoc-02410d4e02a46743f",
          "CidrBlock": "10.0.0.0/16",
          "CidrBlockState": {
            "State": "associated"
          }
        }
      ],
      "IsDefault": false,
      "Tags": [
        {
          "Key": "ManagedBy",
          "Value": "Terraform"
        },
        {
          "Key": "Name",
          "Value": "prod-vpc"
        },
        {
          "Key": "Project",
          "Value": "Assignment-2"
        },
        {
          "Key": "Environment",
          "Value": "prod"
        }
      ],
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "VpcId": "vpc-066d23f8ffc8763f6",
    }
  ]
}
```

aws ec2 describe-subnets


```

@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-subnets
{
  "Subnets": [
    {
      "AvailabilityZoneId": "mec1-az2",
      "MapCustomerOwnedIpOnLaunch": false,
      "OwnerId": "095032391730",
      "AssignIpv6AddressOnCreation": false,
      "Ipv6CidrBlockAssociationSet": [],
      "Tags": [
        {
          "Key": "Name",
          "Value": "Lab8VPC-subnet-public2-me-central-1b"
        }
      ],
      "SubnetArn": "arn:aws:ec2:me-central-1:095032391730:subnet/subnet-00c99c64db5b66abb",
      "EnableDns64": false,
      "Ipv6Native": false,
      "PrivateDnsNameOptionsOnLaunch": {
        "HostnameType": "ip-name",
        "EnableResourceNameDnsARecord": false,
        "EnableResourceNameDnsAAAARecord": false
      },
      "BlockPublicAccessStates": {
        "InternetGatewayBlockMode": "off"
      },
      "SubnetId": "subnet-00c99c64db5b66abb",
      "State": "available",
      "VpcId": "vpc-002a349744ae1a88b",
      "CidrBlock": "10.0.16.0/20",
      "AvailableIpAddressCount": 4091,
      "AvailabilityZone": "me-central-1b",
      "DefaultForAz": false,
      "MapPublicIpOnLaunch": false
    },
    {
      "AvailabilityZoneId": "mec1-az2",

```

aws ec2 describe-instances

```

@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-instances
{
  "Reservations": [
    {
      "ReservationId": "r-0ae645c9ae782e29f",
      "OwnerId": "095032391730",
      "Groups": [],
      "Instances": [
        {
          "Architecture": "x86_64",
          "BlockDeviceMappings": [
            {
              "DeviceName": "/dev/xvda",
              "Ebs": {
                "AttachTime": "2026-01-08T10:29:59+00:00",
                "DeleteOnTermination": true,
                "Status": "attached",
                "VolumeId": "vol-03e4c94bf9d05f23f"
              }
            }
          ],
          "ClientToken": "1ff2f171-0011-4e7c-af97-24e12e8d1bc4",
          "EbsOptimized": true,
          "EnaSupport": true,
          "Hypervisor": "xen",
          "NetworkInterfaces": [
            {
              "Attachment": {
                "AttachTime": "2026-01-08T10:29:58+00:00",
                "AttachmentId": "eni-attach-0e870e732d8592651",
                "DeleteOnTermination": true,
                "DeviceIndex": 0,
                "Status": "attached",
                "NetworkCardIndex": 0
              },
              "Description": "",
              "Groups": [

```

aws ec2 describe-regions

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-regions
{
  "Regions": [
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "ap-south-1",
      "Endpoint": "ec2.ap-south-1.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-north-1",
      "Endpoint": "ec2.eu-north-1.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-west-3",
      "Endpoint": "ec2.eu-west-3.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-west-2",
      "Endpoint": "ec2.eu-west-2.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "eu-west-1",
      "Endpoint": "ec2.eu-west-1.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "ap-northeast-3",
      "Endpoint": "ec2.ap-northeast-3.amazonaws.com"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "RegionName": "ap-northeast-2",
      "Endpoint": "ec2.ap-northeast-2.amazonaws.com"
    }
  ]
}
```

aws ec2 describe-availability-zones

```

OptInStatus: opt-in-not-required,
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-availability-zones
{
  "AvailabilityZones": [
    {
      "OptInStatus": "opt-in-not-required",
      "Messages": [],
      "RegionName": "me-central-1",
      "ZoneName": "me-central-1a",
      "ZoneId": "mec1-az1",
      "GroupName": "me-central-1-zg-1",
      "NetworkBorderGroup": "me-central-1",
      "ZoneType": "availability-zone",
      "GroupLongName": "Middle East (UAE) 1",
      "State": "available"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "Messages": [],
      "RegionName": "me-central-1",
      "ZoneName": "me-central-1b",
      "ZoneId": "mec1-az2",
      "GroupName": "me-central-1-zg-1",
      "NetworkBorderGroup": "me-central-1",
      "ZoneType": "availability-zone",
      "GroupLongName": "Middle East (UAE) 1",
      "State": "available"
    },
    {
      "OptInStatus": "opt-in-not-required",
      "Messages": [],
      "RegionName": "me-central-1",
      "ZoneName": "me-central-1c",
      "ZoneId": "mec1-az3",
      "GroupName": "me-central-1-zg-1",
      "NetworkBorderGroup": "me-central-1",
      "ZoneType": "availability-zone",
      "GroupLongName": "Middle East (UAE) 1"
    }
  ]
}

```

Task 6 — IAM: create group, user, attach policies, create console login & keys

Goal: Practice managing IAM via CLI: create group and user, attach policies, add user to group, create login profile, attach/detach change-password policy, create access keys, and test access via environment variables.

Commands and immediate screenshot request after each step:

1. Create group:

```
aws iam create-group --group-name MyGroupCli
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam create-group --group-name MyGroupCli
{
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPARMIC7HAZPFWJIQF7R",
    "Arn": "arn:aws:iam::095032391730:group/MyGroupCli",
    "CreateDate": "2026-01-09T02:40:08+00:00"
  }
}
@manahilhabib031 → /workspaces/Lab9 (main) $
```

2. Get group details:

aws iam get-group --group-name MyGroupCli

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam get-group --group-name MyGroupCli
{
  "Users": [],
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPARMIC7HAZPFWJIQF7R",
    "Arn": "arn:aws:iam::095032391730:group/MyGroupCli",
    "CreateDate": "2026-01-09T02:40:08+00:00"
  }
}
@manahilhabib031 → /workspaces/Lab9 (main) $
```

3. Create user:

aws iam create-user --user-name MyUserCli

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam create-user --user-name MyUserCli
{
  "User": {
    "Path": "/",
    "UserName": "MyUserCli",
    "UserId": "AIDARMIC7HAZAMI6KUF2K",
    "Arn": "arn:aws:iam::095032391730:user/MyUserCli",
    "CreateDate": "2026-01-09T02:40:50+00:00"
  }
}
@manahilhabib031 → /workspaces/Lab9 (main) $
```

4. Get user details:

aws iam get-user --user-name MyUserCli

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam get-user --user-name MyUserCli
{
  "User": {
    "Path": "/",
    "UserName": "MyUserCli",
    "UserId": "AIDARMIC7HAZAMI6KUF2K",
    "Arn": "arn:aws:iam::095032391730:user/MyUserCli",
    "CreateDate": "2026-01-09T02:40:50+00:00"
  }
}
@manahilhabib031 → /workspaces/Lab9 (main) $
```

5.Add user to group:

```
aws iam add-user-to-group --user-name MyUserCli --group-name MyGroupCli
```

6.See group again:

```
aws iam get-group --group-name MyGroupCli
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam add-user-to-group \
--user-name MyUserCli \
--group-name MyGroupCli
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam get-group --group-name MyGroupCli
{
  "Users": [
    {
      "Path": "/",
      "UserName": "MyUserCli",
      "UserId": "AIDARMIC7HAZAMI6KUF2K",
      "Arn": "arn:aws:iam::095032391730:user/MyUserCli",
      "CreateDate": "2026-01-09T02:40:50+00:00"
    }
  ],
  "Group": {
    "Path": "/",
    "GroupName": "MyGroupCli",
    "GroupId": "AGPARMIC7HAZPFWJIQF7R",
    "Arn": "arn:aws:iam::095032391730:group/MyGroupCli",
    "CreateDate": "2026-01-09T02:40:08+00:00"
  }
}
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

7.List policies that mention EC2:

```
aws iam list-policies \
```

```
--query "Policies[?contains(PolicyName, 'EC2')].{Name:PolicyName}" \
```

```
--output text
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam list-policies \
--query "Policies[?contains(PolicyName, 'EC2')].{Name:PolicyName}" \
--output text
AmazonEC2FullAccess
AmazonEC2ReadOnlyAccess
AmazonElasticMapReduceforEC2Role
AmazonEC2RoleforDataPipelineRole
AmazonEC2ContainerServiceforEC2Role
AmazonEC2ContainerServiceRole
AmazonEC2RoleforAWSCodeDeploy
AmazonEC2RoleforSSM
CloudWatchActionsEC2Access
AmazonEC2ContainerRegistryReadOnly
AmazonEC2ContainerRegistryPowerUser
AmazonEC2ContainerRegistryFullAccess
AmazonEC2ContainerServiceAutoscaleRole
AmazonEC2SpotFleetAutoscaleRole
AWSElasticBeanstalkCustomPlatformforEC2Role
AmazonEC2ContainerServiceEventsRole
AmazonEC2SpotFleetTaggingRole
AWSEC2SpotServiceRolePolicy
AWSServiceRoleForEC2ScheduledInstances
AWSEC2SpotFleetServiceRolePolicy
AWSApplicationAutoscalingEC2SpotFleetRequestPolicy
AWSEC2FleetServiceRolePolicy
AWSAutoScalingPlansEC2AutoScalingPolicy
EC2InstanceConnect
AmazonEC2RolePolicyForLaunchWizard
EC2InstanceProfileForImageBuilder
EC2FleetTimeShiftableServiceRolePolicy
AmazonEC2RoleforAWSCodeDeployLimited
EC2InstanceProfileForImageBuilderECRContainerBuilds
AWSApplicationMigrationEC2Access
AWSEC2CapacityReservationFleetRolePolicy
EC2FastLaunchServiceRolePolicy
AmazonSSMManagedEC2InstanceDefaultPolicy
```

8. Get ARN for AmazonEC2FullAccess (example query):

```
aws iam list-policies --query
'Policies[?PolicyName=='AmazonEC2FullAccess'].{Name:PolicyName, ARN:Arn}' --output
table
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam list-policies \
--query 'Policies[?PolicyName=='AmazonEC2FullAccess'].{Name:PolicyName, ARN:Arn}' \
--output table
-----
|                               ListPolicies                               |
+-----+-----+-----+-----+
|                               ARN                               | Name |
+-----+-----+-----+-----+
| arn:aws:iam::aws:policy/AmazonEC2FullAccess | AmazonEC2FullAccess |
+-----+-----+-----+-----+
@manahilhabib031 → /workspaces/Lab9 (main) $
```

9. Attach policy to group (use the ARN you retrieved):

```
aws iam attach-group-policy \
```

--group-name MyGroupCli \

--policy-arn arn:aws:iam::aws:policy/EXAMPLEPolicyName

10. List attached policies for the group:

aws iam list-attached-group-policies --group-name MyGroupCli

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam attach-group-policy \
--group-name MyGroupCli \
--policy-arn arn:aws:iam::aws:policy/AmazonEC2FullAccess
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam list-attached-group-policies --group
-name MyGroupCli
{
  "AttachedPolicies": [
    {
      "PolicyName": "AmazonEC2FullAccess",
      "PolicyArn": "arn:aws:iam::aws:policy/AmazonEC2FullAccess"
    }
  ]
}
```

11. Create a console login profile for the user:

aws iam create-login-profile \

--user-name MyUserCli \

--password <PASSWORD_VALUE> \

--password-reset-required

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam create-login-profile \
--user-name MyUserCli \
--password TempPassword@123 \
--password-reset-required
{
  "LoginProfile": {
    "UserName": "MyUserCli",
    "CreateDate": "2026-01-09T02:44:58+00:00",
    "PasswordResetRequired": true
  }
}
```

12. If the user cannot change password, attach IAMUserChangePassword to the group temporarily:

aws iam attach-group-policy --group-name MyGroupCli --policy-arn
arn:aws:iam::aws:policy/IAMUserChangePassword

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam detach-group-policy \
--group-name MyGroupCli \
--policy-arn arn:aws:iam::aws:policy/IAMUserChangePassword
```

After the user logs in and resets password, detach that policy:

```
aws iam detach-group-policy --group-name MyGroupCli --policy-arn
arn:aws:iam::aws:policy/IAMUserChangePassword
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam detach-group-policy \
--group-name MyGroupCli \
--policy-arn arn:aws:iam::aws:policy/IAMUserChangePassword
```

13. Create access keys for the user (save AccessKeyId and SecretAccessKey securely):

```
aws iam create-access-key --user-name MyUserCli
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam create-access-key --user-name MyUserC
Cli
{
  "AccessKey": {
    "UserName": "MyUserCli",
    "AccessKeyId": "AKIA...",
    "Status": "Active",
    "SecretAccessKey": "tRRruxrg...",
    "CreateDate": "2026-01-09T02:47:51+00:00"
  }
}
```

14. List access keys:

```
aws iam list-access-keys --user-name MyUserCli
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam list-access-keys --user-name MyUserC
li
{
  "AccessKeyMetadata": [
    {
      "UserName": "MyUserCli",
      "AccessKeyId": "AKIA...",
      "Status": "Active",
      "CreateDate": "2026-01-09T02:47:51+00:00"
    }
  ]
}
@manahilhabib031 → /workspaces/Lab9 (main) $
```

15. (Don't) Delete access key:

```
aws iam delete-access-key --user-name MyUserCli --access-key-id <AccessKeyId> # Don't run
this command
```

16. Use environment variables to authenticate as that user in the Codespace:

```
export AWS_ACCESS_KEY_ID=<YOUR_ACCESS_KEY_ID>
```

```
export AWS_SECRET_ACCESS_KEY=<YOUR_SECRET_ACCESS_KEY>
```

```
printenv | grep AWS_
```

```
aws iam get-user --user-name MyUserCli # may fail if no permissions
```


exit # to clear exports

```
@manahilhabib031 → /workspaces/Lab9 (main) $ export AWS_ACCESS_KEY_ID=AKIARMIC7HAZAMI6KUF2K
M
@manahilhabib031 → /workspaces/Lab9 (main) $ export AWS_SECRET_ACCESS_KEY=tRRruxrg0V7NYpk
BZd3XVfe" h
@manahilhabib031 → /workspaces/Lab9 (main) $ printenv | grep AWS_
AWS_SECRET_ACCESS_KEY=tRRruxrg0V7NYpk
AWS_ACCESS_KEY_ID=AKIARMIC7HAZAMI6KUF2K
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam get-user --user-name MyUserCli

An error occurred (AccessDenied) when calling the GetUser operation: User: arn:aws:iam::095032391730:user/MyUserCli is not authorized to perform: iam:GetUser on resource: user MyUserCli because no identity-based policy allows the iam:GetUser action
@manahilhabib031 → /workspaces/Lab9 (main) $
```

After clearing or switching credentials, repeat get-user and save:

```
@manahilhabib031 → /workspaces/Lab9 (main) $ exit
logout
Connection to localhost closed.
shell closed: exit status 254

C:\Users\HABIB TARIQ>aws iam get-user --user-name MyUserCli
{
  "User": {
    "Path": "/",
    "UserName": "MyUserCli",
    "UserId": "AIDARMIC7HAZAMI6KUF2K",
    "Arn": "arn:aws:iam::095032391730:user/MyUserCli",
    "CreateDate": "2026-01-09T02:40:50+00:00"
  }
}

C:\Users\HABIB TARIQ>
```

Task 7 — Filters: query with filters to find instances and their attributes

Goal: Use filters and queries to list specific instances and attributes.

Examples (run each and take a screenshot immediately after):

aws ec2 describe-instances \

--filters "Name=tag:Name,Values=MyServer" \

--query "Reservations[*].Instances[*].PublicIpAddress" \

--output text

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-instances \
--filters "Name=tag:Name,Values=MyServer" \
--query "Reservations[*].Instances[*].PublicIpAddress" \
--output text
40.172.151.208
```

aws ec2 describe-instances \

--filters "Name=instance-type,Values=t3.micro" \

--query "Reservations[].Instances[].InstanceId" \

--output table

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-instances \
--filters "Name=instance-type,Values=t3.micro" \
--query "Reservations[].Instances[].InstanceId" \
--output table
```

DescribeInstances
i-080271458b4e9b2d1
i-0ffbcddcd6910d18a4

aws ec2 describe-instances \

--filters "Name=subnet-id,Values=subnet-0600df5fa8ce60857" \

--query "Reservations[*].Instances[*].InstanceId" \

--output table

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-instances \
--filters "Name=subnet-id,Values=subnet-0f284e40a191a3e62" \
--query "Reservations[*].Instances[*].InstanceId" \
--output table
```

DescribeInstances
i-0ffbcddcd6910d18a4

aws ec2 describe-instances \

--filters "Name=vpc-id,Values=vpc-06be85cd81b657192" \

--query "Reservations[*].Instances[*].InstanceId" \

--output table

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-instances \
--filters "Name=vpc-id,Values=vpc-021a5e129ca204b3f" \
--query "Reservations[*].Instances[*].InstanceId" \
--output table
```

DescribeInstances
i-0ffbcdcd6910d18a4

Task 8 — Use --query to format outputs for reporting

Goal: Extract useful fields in table format for reporting.

Examples (run each and take a screenshot immediately after):

```
aws ec2 describe-instances \
```

```
--filters "Name=tag:Name,Values=MyServer" \
```

```
--query
```

```
"Reservations[*].Instances[*].[InstanceId,PublicIpAddress,Tags[?Key=='Name'].Value|[0]]" \
```

```
--output table
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-instances \
--filters "Name=tag:Name,Values=MyServer" \
--query "Reservations[*].Instances[*].[InstanceId,PublicIpAddress,Tags[?Key=='Name'].Value|[0]]" \
--output table
```

DescribeInstances		
i-0ffbcdcd6910d18a4	40.172.151.208	MyServer

```
@manahilhabib031 → /workspaces/Lab9 (main) $
```

```
aws ec2 describe-instances \
```

```
--query "Reservations[*].Instances[*].[InstanceId,State.Name]" \
```

```
--output table
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-instances \
--query "Reservations[*].Instances[*].[InstanceId,State.Name]" \
--output table
```

DescribeInstances	
i-080271458b4e9b2d1	terminated
i-0ffbcdcd6910d18a4	running

```
@manahilhabib031 → /workspaces/Lab9 (main) $
```

```
aws ec2 describe-instances \
```

```
--query "Reservations[*].Instances[*].[InstanceId,InstanceType,Placement.AvailabilityZone]" \
```

```
--output table
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-instances \
--query "Reservations[*].Instances[*].[InstanceId,InstanceType,Placement.AvailabilityZone]" \
--output table
-----+-----+-----+
| DescribeInstances |
+-----+-----+-----+
| i-080271458b4e9b2d1 | t3.micro | me-central-1c |
| i-0ffbcddcd6910d18a4 | t3.micro | me-central-1c |
+-----+-----+-----+
@manahilhabib031 → /workspaces/Lab9 (main) $
```

Cleanup — Remove resources to avoid charges

After verification, terminate and delete everything you created in AWS. Capture screenshots for each step.

Cleanup steps and required screenshots (take each screenshot immediately after running the command):

1. Terminate instances:

```
aws ec2 terminate-instances --instance-ids i-EXAMPLE1234567890
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 terminate-instances --instance-ids i-0ffbcddcd6910d18a4
{
  "TerminatingInstances": [
    {
      "InstanceId": "i-0ffbcddcd6910d18a4",
      "CurrentState": {
        "Code": 32,
        "Name": "shutting-down"
      },
      "PreviousState": {
        "Code": 16,
        "Name": "running"
      }
    }
  ]
}
```

2. Delete EBS volumes & snapshots (if any):

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-volumes --output table
-----+-----+
| DescribeVolumes |
+-----+-----+
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-snapshots --owner-ids self
--output table
-----
|DescribeSnapshots|
+-----+
```

3.Delete security group and key pair:

```
aws ec2 delete-security-group --group-id sg-EXAMPLE1234567890
```

```
aws ec2 delete-key-pair --key-name MyED25519Key
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 delete-key-pair --key-name MyED25519
Key
{
  "Return": true,
  "KeyPairId": "key-087e258ebae802e06"
}
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

4.Remove IAM users, access keys, groups:

```
aws iam delete-access-key --user-name MyUserCli --access-key-id <AccessKeyId>
```

```
aws iam delete-login-profile --user-name MyUserCli
```

```
aws iam remove-user-from-group --user-name MyUserCli --group-name MyGroupCli
```

```
aws iam delete-user --user-name MyUserCli
```

```
aws iam detach-group-policy --group-name MyGroupCli --policy-arn
arn:aws:iam::aws:policy/EXAMPLEPolicyName
```

```
aws iam delete-group --group-name MyGroupCli
```

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam delete-access-key --user-name MyUs
erCli --access-key-id AKIARMIC7HAZLE62WGXM
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam delete-login-profile --user-name MyU
serCli
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam remove-user-from-group \
--user-name MyUserCli \
--group-name MyGroupCli
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam delete-user --user-name MyUserCli
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam detach-group-policy \
--group-name MyGroupCli \
--policy-arn arn:aws:iam::aws:policy/AmazonEC2FullAccess
@manahilhabib031 → /workspaces/Lab9 (main) $ aws iam delete-group --group-name MyGroupCli
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```

5.Final verification (billing/resource groups):

```
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 delete-key-pair --key-name myweb
{
  "Return": true,
  "KeyPairId": "key-0acde7469035039ae"
}
@manahilhabib031 → /workspaces/Lab9 (main) $ aws ec2 describe-volumes --output table
-----
|DescribeVolumes|
+-----+
@manahilhabib031 → /workspaces/Lab9 (main) $ |
```